Programação 2

Aula 9 - Erros: tratamento de exceções e depuração da execução

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

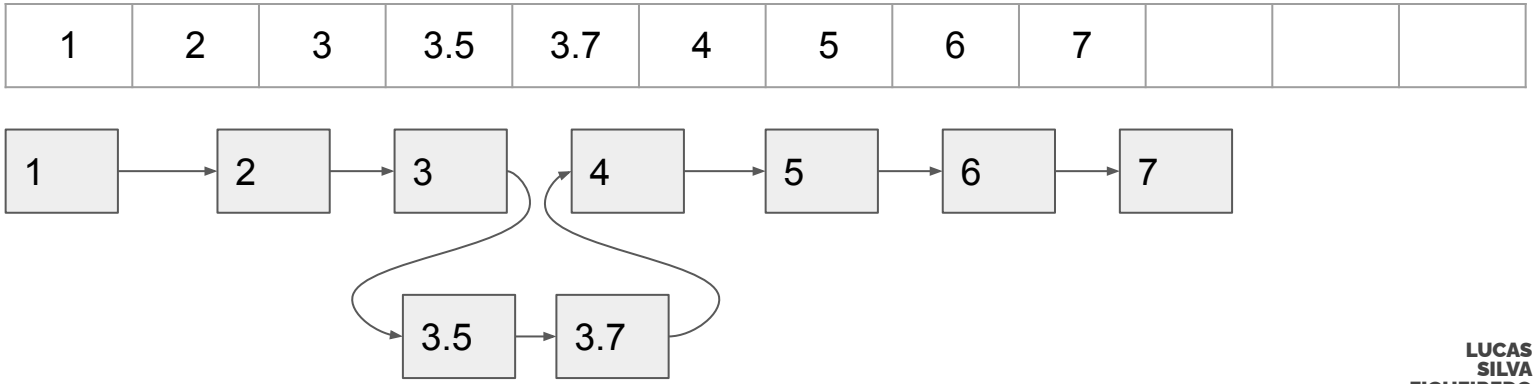
Recapitulando

Listas Ligadas (*Linked Lists*)

Outro conceito para **conectar elementos de um mesmo tipo** é o de Listas Ligadas

Listas Ligadas é uma **forma mais dinâmica de representar coleções** de elementos, facilitando por exemplo a

adição de um elemento na posição do meio da lista



Árvores

Ao lado segue um exemplo de árvore.

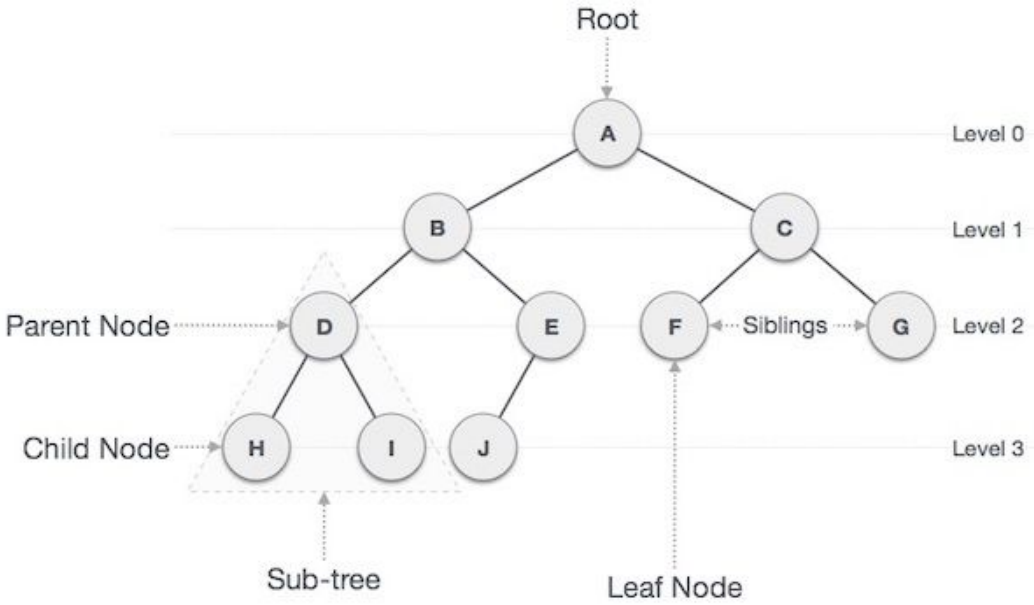
Root denota a raiz da árvore.

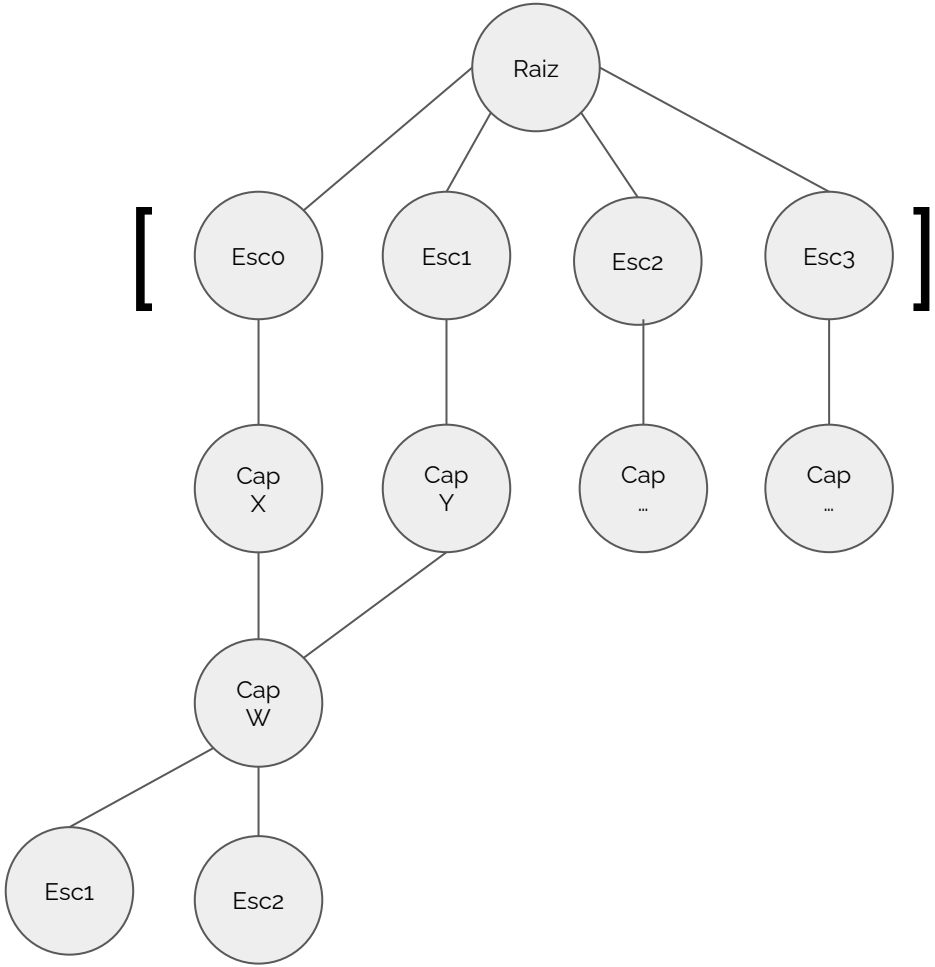
Cada elemento é comumente chamado de nó (**Node**).

Existe uma hierarquia evidente, nomeando as relações entre os nós como relações de parentescos

Parent Node,
Child Node,
Siblings

Os nós que não tem filhos são comumente chamados de folhas (**Leaf**)



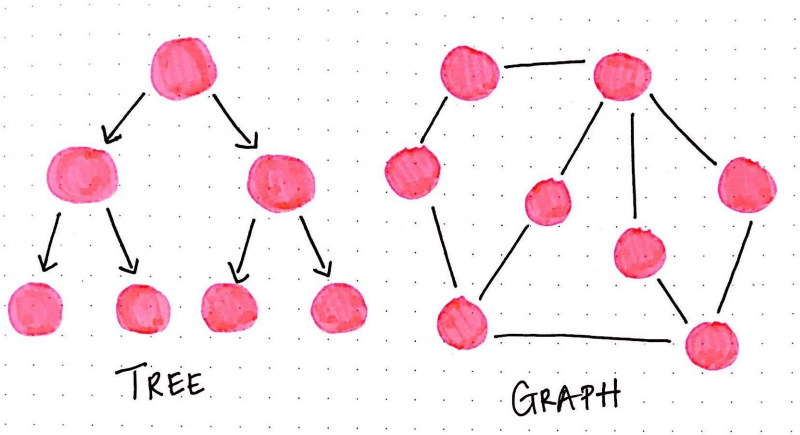


LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Grafos

Grafos por sua vez são a generalização das árvores. Em grafos não existe a ideia de hierarquia

Ao lado segue uma imagem comparando o exemplo de uma árvore com um grafo



Conceito de Hoje

Conceito de Hoje

Erros: tratamento de exceções e depuração da execução

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Introdução

Problema

Motivação

História e Pessoas

1) On average, a developer creates 70 bugs per 1000 lines of code (!)

2) 15 bugs per 1,000 lines of code find their way to the customers

3) Fixing a bug takes 30 times longer than writing a line of code

4) 75% of a developer's time is spent on debugging (1500 hours a year!)

5) In the US alone, $113B is spent annually on identifying & fixing product defects

This is what your developers are doing 75% of the time (coralogix.com)

An Exploratory Study of Debugging Episodes

Abdulaziz Alaboudi
George Mason University
Fairfax, Virginia, USA
aalaboud@gmu.edu

Thomas D. LaToza
George Mason University
Fairfax, Virginia, USA
tlatoza@gmu.edu

5 May 2021

Abstract

Many studies have long investigated how developers debug, shaping our understanding of debugging and helping motivate the creation of more effective tools. However, less is known about the typical progression of debugging in real world settings. In this study, we focus on characterizing debugging episodes from the moment at which developers first encounter a defect to the moment at which it is resolved. We investigate the typical duration and frequency of debugging episodes and the typical activities which occur. We observed developers by watching professional developers at work in live-streamed programming sessions. Using this data source, we curated 15 sessions in which 11 professional developers worked for 30 hours. We then systematically coded the debugging episodes in the lab or through the analysis of log data from instrumented development environments. Lab settings allow researchers to observe developers while debugging after receiving a defect report. However, lab settings are inherently artificial, reflecting an unfamiliar context in which developers work on unfamiliar code. While log data enables understanding debugging in naturalistic contexts, it is inherently limited in its ability to reveal developer intent. While log data can effectively answer questions about how frequently various tools are used, it is much more difficult to answer questions developers' intent when using each tool.

In this paper, we conducted the first *exploratory study* of debugging episodes in a naturalistic setting. A debugging episode begins when a developer first encounters a defect while programming

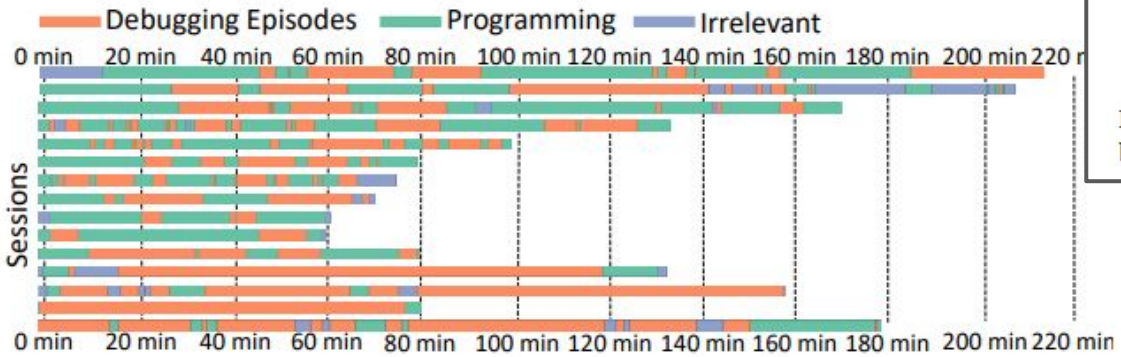


Figure 3: Debugging episodes and programming in the 15 sessions.

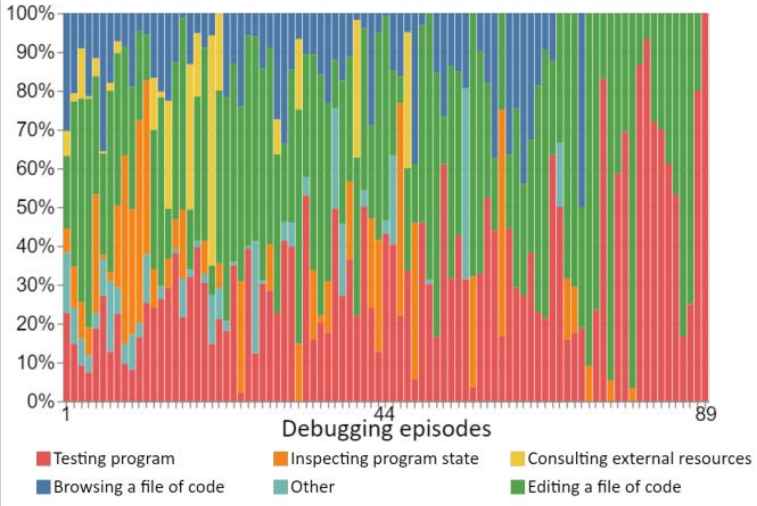


Figure 5: The fraction of time spent in each activity per debugging episode.

LEARNABLE PROGRAMMING

Designing a programming system for understanding programs

Bret Victor / September 2012

Here's a trick question: *How do we get people to understand programming?*

Khan Academy recently launched an online environment for learning to program. It offers a set of tutorials based on the JavaScript and Processing languages, and features a "live coding" environment, where the program's output updates as the programmer types.

Because my work was cited as an inspiration for the Khan system, I felt I should respond with two thoughts about learning:

- **Programming is a way of thinking, not a rote skill.** Learning about "for" loops is not learning to program, any more than learning about pencils is learning to draw.
- **People understand what they can see.** If a programmer cannot see what a program is doing, she can't understand it.

Thus, the goals of a programming system should be:

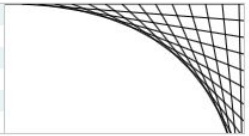
- to support and encourage powerful ways of thinking
- to enable programmers to see and understand the execution of their programs

A live-coding Processing environment addresses neither of these goals. JavaScript and Processing are poorly-designed languages that support weak ways of thinking, and ignore decades of learning about learning. And live coding, as a standalone feature, misses the point.

Alan Perlis wrote, *"To understand a program, you must become both the machine and the program."* This view is a mistake, and it is this widespread and virulent mistake that keeps programming a difficult and obscure art. A person is not a machine, and should not be forced to think like one.

To make the flow more sane for a learner, the loop can be rewritten using "while":

```
var shift = 0;
while (shift <= 200) {
    line(shift, 0, 200, shift);
    shift += 14;
}
```

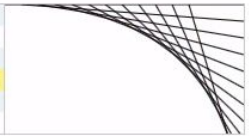


Now, the control flow must be made *tangible*. We must put the execution of the program into the programmer's hand, let her feel that it is a *real thing*, let her *own* it.

In the following example, the programmer uses a slider to scrub through the execution:

step 36

```
var shift = 0;
while (shift <= 200) {
    line(shift, 0, 200, shift);
    shift += 14;                    Set the "shift" variable to 168.
}
```



This control allows the programmer to move around the loop at her own pace, and understand what is happening at each step. She can go backwards and forwards, dwell in difficult areas, and compare what is happening at different times. She can study how the output is built up over time, instead of seeing it magically appear all at once.

Learnable Programming (worrydream.com)

Computer programming | Computing | Khan Academy

Apresentação

Definições

Exemplos Ilustrativos

Erros

Existem 3 tipos de erro em linguagens como Java

- Erros de compilação
- Erros de execução
- Erros de lógica/comportamento

Erros

Existem 3 tipos de erro em linguagens como Java

- Erros de compilação
- Erros de execução
- Erros de lógica/comportamento

Erros de Compilação

Erros de Compilação

No processo de construção do código binário

Falhas de sintaxe, por exemplo:

- **nomes errados** de variáveis, classes, métodos, etc
- **tipos** não declarados
- **; faltando**
- **ordenação das palavras reservadas**

Erros de inclusão de arquivos e bibliotecas

Erros de incompatibilidade de versões do compilador

Erros de Execução

Erros de Execução

No processo de rodar (runtime) do código binário

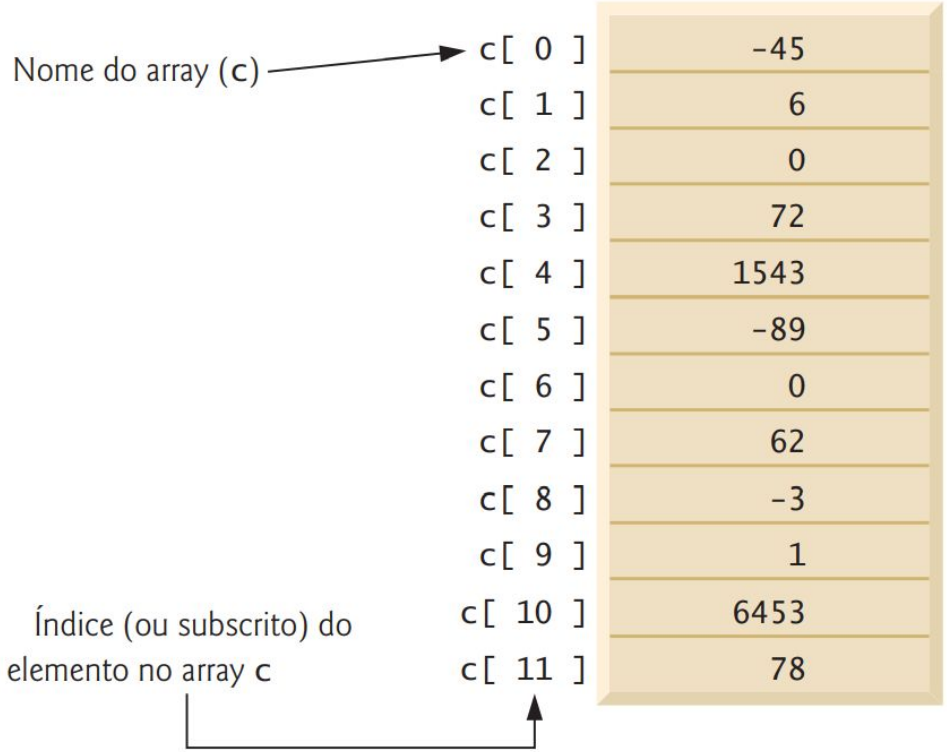
Em Java são chamados de **exceções**
exemplos são:

- ArrayIndexOutOfBoundsException
- ClassNotFoundException
- FileNotFoundException
- NullPointerException

Erros de Execução

```
Julia estava correndo pela floresta e se deparou com um penhasco. O que ela fará?
[Devido aos acontecimentos recentes, a energia de Julia foi a 90]
- Pula
- Se esconde
. . .
Se esconde
Ao se esconder ela vê quatro lobos que estavam perseguindo ela.
Claramente um deles é mais imponente, ele mentalmente apelida ele de Alfa
Aparentemente os lobos se perderem, e agora?
[Devido aos acontecimentos recentes, a energia de Alfa foi a 185]
- Sobe numa árvore
- Corre para os lobos
. . .
Corre para os lobos
[Final 3] Ao correr na direç?o dos lobos, Julia percebe que
Alfaestava na verdade latindo para avisar de um penhasco logo a frente.
Juliafica feliz em perceber o cuidado de Alfa e assim
ambos começam uma amizade bonita.
[Devido aos acontecimentos recentes, a energia de Julia foi a -10]
Exception in thread "main" java.lang.NullPointerException
        at Capitulo.mostrar(Capitulo.java:33)
        at Capitulo.mostrar(Capitulo.java:43)
        at Capitulo.mostrar(Capitulo.java:43)
        at Test.main(Test.java:67)
Press any key to continue . . .
```

Arrays: Acesso




```
c[12] = 12;
```

o que aconteceu na linha acima?

Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException:
Index 12 out of bounds for length 12
at Main.main(Main.java:12)

Exceções

Aplicações Java pode ser multithread (rodar em mais de um processador), a mensagem de erro deixa claro qual foi a thread

Existem vários tipos de exceções. A que vimos é especificamente sobre acessos em arrays.

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException:
Index 12 out of bounds for length 12
        at Main.main(Main.java:12)
```

As mensagens de exceções indicam qual foi a linha de código que lançou a exceção.

Exceções: Tratamento com **try** -> **catch**

Exceções causam o **encerramento da aplicação**

Java permite capturar as exceções antes disso ocorrer e tratá-las usando a combinação dos blocos de

- Tentativa de execução (**try**)
- Captura e tratamento de exceções (**catch**)

Exceções: Tratamento com **try** -> **catch**

```java
// acesso inválido
int[] c = new int[12];
c[12] = 12;

// acréscimo de um ponto para todos
for(int i = 0; i < c.length; i++)
{
    c[i] += 1;
}
```

```
Exception in thread "main"
java.lang.ArrayIndexOutOfBoundsException:
Index 12 out of bounds for length 12
    at Main.main(Main.java:10)
```

```java
try
{
    // acesso inválido
    int[] c = new int[12];
    c[12] = 12;

    // acréscimo de um ponto para todos
    for(int i = 0; i < c.length; i++)
    {
        c[i] += 1;
    }
}
catch(ArrayIndexOutOfBoundsException exception)
{
    System.out.println("você acessou uma posição inválida");
    System.out.println(exception);
}
```

```
você acessou uma posição inválida
java.lang.ArrayIndexOutOfBoundsException:
Index 12 out of bounds for length 12
```

Erros de Lógica/Comportamento

Erros de Lógica/Comportamento

O código a seguir compila e roda! Mas não funciona com o esperado

```
// executando a potencia com a repetição usando while
int i = 1;
int numeroA = 2;
int potencia = 5;

while (i < potencia)
{
  numeroA = numeroA * numeroA;
  i++;
}
```

Depuração/Debugging

File Edit Selection View Go Run Terminal Help

Test.java - FirstProject - Visual Studio Code

RUN AND DEBUG No Configurati

VARIABLES

Local

```
args: String[0]@7
escaneador: (...)
protagonista: Personagem@9
```

WATCH

CALL STACK

Thread [Attach Listener]	RUNNING
Thread [Signal Dispatcher]	RUNNING
Thread [Finalizer]	RUNNING
Thread [Reference Handler]	RUNNING
Thread [main]	PAUSED ON BREAKPOINT
Test.main(String[]) Test.java 11:1	

BREAKPOINTS

- [ ] Uncaught Exceptions
- [ ] Caught Exceptions
- [x] Test.java src 11

Test.java 1 | App.java | Pessoa.java

src > Test.java > Test > main(String[])

```
1  import java.util.ArrayList;
2  import java.util.Scanner;
3
4  public class Test
5  {
       Run | Debug
6      public static void main(String[] args)  args = String[0]@7
7      {
8          Scanner escaneador = new Scanner(System.in); escaneador = Scanner@8
9
10         Personagem protagonista = new Personagem("Julia"); protagonista = Personagem@9
11         Personagem antagonista = new Personagem("Alfa", 200);
12
13         // A partir da classe Capitulo, criar os objetos
14         Capitulo capitulo1 =
15             new Capitulo(protagonista.nome +
16             " estava correndo pela floresta e se deparou com um penhasco. O que ela fará?",
17             new String[]{"Pula", "Se esconde"},
18             protagonista, antagonista,
19             -10, 0, escaneador);
20
21         Capitulo capitulo11 =
22             new Capitulo(
23             "[Final 1] Ao pular do penhasco ela acaba se sacrificando",
24             new String[]{},
25             protagonista, antagonista,
26             -100, 0, escaneador);
27
28         Capitulo capitulo12 =
29             new Capitulo(
30             "Ao se esconder ela vê quatro lobos que estavam perseguindo ela.\n"+
31             "Claramente um deles é mais imponente, ele mentalmente apelida ele de "+ antagonista.nome+"\n"+
32             "Aparentemente os lobos se perderem, e agora?",
33             new String[]{"Sobe numa árvore", "Corre para os lobos"},
34             protagonista, antagonista,
35             0, -15, escaneador);
```

OUTPUT TERMINAL 2 DEBUG CONSOLE

Filter (e.g. text, !exclude)

Revisão Técnica do Projeto: recupera até 50% dos pontos

segue a especificação referente a recuperação de pontos de cada mini-projeto

1. Crie um programa que simula um "**livro interativo**", com pelo menos duas **escolhas consecutivas**, e **três finais diferentes**.

2. Crie uma **classe Personagem** com os atributos **String** nome e **int** energia; crie dois objetos (protagonista e antagonista) usando o construtor dessa classe; crie dois capítulos em que devido a história suas energias mudam

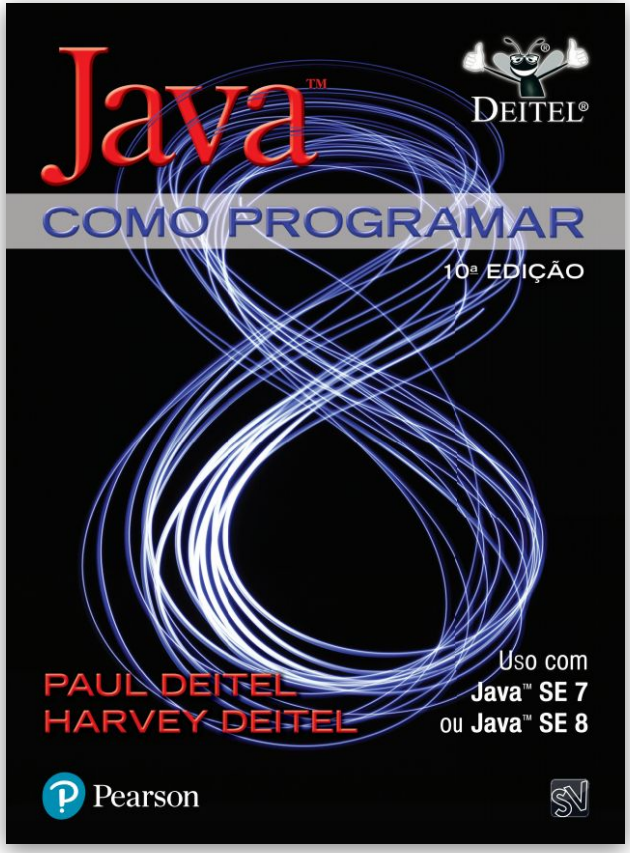
3. Crie uma classe **Capitulo** com os atributos representando o texto, as escolhas, os personagens e a alteração na energia deles; crie um método **void mostrar()** para mostrar o texto e as escolhas no console e em seguida capturar a escolha do jogador

4. Use um **array** ou **ArrayList** de escolhas na classe Capitulo

5. Crie uma **classe Escolha** com os atributos: **String** texto; **Capitulo** proximo; use o conceito de **árvore** (ou grafo se quiser ter loops ou convergências da história) para representar seu livro interativo, em que exista um capítulo raiz, e a partir das escolhas dele outros capítulos serão apresentados; **no método main o único método chamado dos capítulos deve ser o método mostrar do capítulo raiz**

Referências

Java - Como Programar (10a Edição)

Ferramenta:

Eclipse: eclipse.org/downloads